

Consolidated Evaluation Report

Total Students	11
Evaluated Students	11
Average Score	13.24 / 20
Highest Score	17.6
Lowest Score	9.8
Plagiarism Cases (≥ 0.80)	1

Student Coverage Overview

Roll Number	Score (/20)	Commits	Public	README
711725UAD136	13.0	8	Yes	Yes
711725UAD202	12.0	12	Yes	Yes
711725UAM123	17.0	7	Yes	Yes
711725UAM251	12.0	4	Yes	Yes
711725UCS149	14.0	1	Yes	Yes
711725UCS247	15.4	21	Yes	Yes
711725UEC163	17.6	7	Yes	Yes
711725UEC233	10.4	2	Yes	No
711725UIT142	13.6	5	Yes	Yes
711725UIT219	10.8	5	Yes	Yes
711725UIT222	9.8	6	Yes	Yes

Student Evaluation Report

Roll Number: 711725UAD136

Repository: <https://github.com/gharshavarhinia-ship-it/miniProjectSourceCode>

Metric	Value
Commit Count	8
Public Repository	Yes
README Present	Yes

Final Score: 13.0 / 20

Overall Remarks: The student has made exceptional modifications, particularly with the implementation of a robust file migration system ('migrateFileIfNeeded') which demonstrates advanced understanding of data persistence and schema evolution. The addition of a 'searchByName' feature significantly enhances usability. The integration of the new 'slot' field is thorough across all relevant functions, showing strong code comprehension and attention to detail. The effort put into these modifications is highly commendable.

GENERAL

Criteria	Score	Remarks
Self Effort	5	Demonstrated exceptional effort by implementing a complex file migration system, adding a new search feature, and thoroughly integrating the new 'slot' field across the application.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Shows excellent understanding of file I/O, data structures, and the practical necessity of data migration for robust applications, as well as common user requirements for data searching.
Added Functionality Ideas	5	The idea of implementing a schema migration strategy (adding 'slot' and converting old data) is a significant and well-conceived functionality idea. 'searchByName' is also a clear, valuable addition.
Code Comprehension	5	The ability to identify the need for and implement file format migration, correctly handle struct size differences, and integrate the 'slot' field into multiple existing functions showcases a deep understanding of the provided codebase.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	The 'migrateFileIfNeeded' function is a crucial code improvement, adding robustness, backward compatibility, and data integrity by ensuring the file format and 'slot' numbers are always correct. Updated display formats also improve usability.
Functional Decomposition	5	The complex migration logic is well-encapsulated in 'migrateFileIfNeeded'. The 'searchByName' functionality is also appropriately separated into its own function.

Criteria	Score	Remarks
Memory Optimization	0	No explicit memory optimization observed.
Speed Optimization	0	No explicit speed optimization observed. The file migration process involves a full file rewrite, which can be slow but is necessary for the task.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	The 'searchByName' function is a new, practical feature that significantly enhances the application's search capabilities.
Requirement Translation	10	Successfully translated the implicit requirement for data consistency and future-proofing (related to adding the 'slot' field) into a robust migration solution. Also translated user search needs into the 'searchByName' feature.
Added Functionality Simple	10	Displaying the 'slot' field in existing listing and search functions, and the 'Warning Low Balance!' message are simple yet effective enhancements to user feedback and data visibility.
Added Functionality Advanced	10	The 'migrateFileIfNeeded' function represents an advanced functionality, demonstrating sophisticated handling of file format changes, data conversion, and file system operations (temp files, rename, remove).

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UAD202

Repository: <https://github.com/Mrudul-P-Manesh/miniProjectSourceCode>

Metric	Value
Commit Count	12
Public Repository	Yes
README Present	Yes

Final Score: 12.0 / 20

Overall Remarks: The student has made significant and commendable modifications to the bank management system. Key contributions include the addition of a 'phone' field to client data, the implementation of robust input validation throughout the application, and the creation of three new, well-defined functions: `searchByName`, `calculateTotals`, and `wipeDatabase`. The `wipeDatabase` function is particularly well-handled, including a warning, user confirmation, and the clearing of associated text files. The code demonstrates excellent functional decomposition, strong comprehension of file I/O, and a clear understanding of practical system requirements. Improvements to the user interface through console clearing and a formatted menu also enhance the overall experience. While there are no advanced optimizations or complex algorithmic innovations, the breadth and quality of the added functionality are outstanding for this type of project.

GENERAL

Criteria	Score	Remarks
Self Effort	5	The student has demonstrated significant self-effort by implementing multiple new features, robust input validation, and UI enhancements. This goes well beyond a basic template.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Demonstrates strong understanding of bank management system requirements by adding relevant features like search by name, total balance calculation, and a phone number field. The use of binary file I/O for persistent storage also shows good domain knowledge in C applications.
Added Functionality Ideas	5	The ideas for added functionality are practical and directly enhance the system's utility: searching by name, calculating bank totals, and providing a database wipe mechanism with confirmation are all valuable additions. The integration of a phone number field is also a good functional idea.
Code Comprehension	5	The student effectively uses C file I/O functions (<code>fopen</code> , <code>fread</code> , <code>fwrite</code> , <code>rewind</code> , <code>fseek</code>), string manipulation (<code>strcmp</code>), and control structures. Input validation loops are correctly implemented across various input points, indicating a thorough understanding of robust input handling. While a minor detail in <code>calculateTotals</code> might count inactive accounts, the overall comprehension of implementing the features is sound.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	A major code improvement is the widespread implementation of robust input validation using <code>`while (scanf(...) != N)`</code> loops with buffer clearing (<code>`while (getchar() != '\n');`</code>). This significantly enhances program stability and user experience. UI improvements like <code>`system('cls')`</code> and a well-formatted menu also contribute to a better user interface.
Functional Decomposition	10	Excellent functional decomposition is evident with the creation of distinct functions like <code>`searchByName`</code> , <code>`calculateTotals`</code> , and <code>`wipeDatabase`</code> . These functions are well-scoped and encapsulate specific logical operations, making the code modular and readable.
Memory Optimization	0	No explicit memory optimization techniques were observed. The <code>`clientData`</code> struct uses fixed-size arrays, which is a standard design choice rather than an optimization.
Speed Optimization	0	No explicit speed optimization techniques were observed. Operations like <code>`searchByName`</code> perform a linear scan of the file, which is simple but not optimized for very large datasets.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	The addition of a phone number field, search by name functionality, and the ability to calculate bank totals (active accounts and total balance) are all basic yet essential features for a bank management system. The database wipe functionality is also a fundamental administrative feature.
Requirement Translation	10	The student successfully translated common requirements of a bank management system into functional code, including data storage enhancements (phone number), reporting (totals), querying (search by name), and administrative tasks (wipe database). The robust input validation addresses an implicit but crucial requirement for reliable software.
Added Functionality Simple	10	The implemented functions (<code>`searchByName`</code> , <code>`calculateTotals`</code> , <code>`wipeDatabase`</code>) directly provide new, tangible capabilities to the system. The systematic application of input validation across all user inputs also represents a significant and simply added functionality that enhances the program's quality.
Added Functionality Advanced	0	While well-implemented, the added functionalities are standard for a file-based CRUD application. There are no advanced data structures, complex algorithms, or sophisticated error handling mechanisms beyond basic input validation.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UAM123

Repository: <https://github.com/dharaneesh-sys/miniProjectSourceCode>

Metric	Value
Commit Count	7
Public Repository	Yes
README Present	Yes

Final Score: 17.0 / 20

Overall Remarks: The student has significantly enhanced the base account management system by introducing several new and well-implemented features. These include a dedicated single account display, an ATM-style debit transaction with robust balance checks, and a valuable function to export sorted account data to a text file. The use of helper functions like ``get_positive_double`` and the clear functional decomposition for new logic are commendable. While the sorting algorithm (Bubble Sort) is basic and not optimized for scalability, and there's a minor undeclared variable in a presumed template function (``writePtr`` in ``text_file``), the overall effort and quality of the added modifications demonstrate a strong understanding of C programming, file I/O, and problem-solving. This is an excellent submission.

GENERAL

Criteria	Score	Remarks
Self Effort	5	The student has made significant self-effort by adding multiple new features, helper functions, and enhancing existing logic, demonstrating a strong initiative beyond the base template.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Demonstrates excellent understanding of the problem domain by implementing robust business logic for transactions (e.g., preventing negative balances, insufficient funds for debit) and managing account records effectively.
Added Functionality Ideas	5	The ideas for new functionalities (single account display, dedicated debit transaction, sorted reports) are practical, highly useful, and enhance the system's utility significantly.
Code Comprehension	5	The student shows a thorough understanding of the existing codebase by correctly integrating new functions with file I/O operations (<code>`fseek`</code> , <code>`fread`</code> , <code>`fwrite`</code>), utilizing existing helper functions, and maintaining the overall program structure.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	Good code improvements include the creation of <code>`get_positive_double`</code> for better input validation, the negative balance check in <code>`update_record`</code> , and structured error handling for file operations in new features. The added features themselves are well-structured.

Criteria	Score	Remarks
Functional Decomposition	10	Excellent functional decomposition. New functionalities like <code>`display_account`</code> , <code>`debit_transaction`</code> , <code>`get_positive_double`</code> , <code>`compare_names`</code> , and <code>`write_sorted_text_file`</code> are all well-separated into distinct, logical functions, greatly improving modularity and readability.
Memory Optimization	5	For the defined <code>`MAX_RECORDS`</code> (100), loading all records into an array for sorting is acceptable. No specific memory optimizations are evident, but also no significant memory waste for the given scale.
Speed Optimization	5	The <code>`write_sorted_text_file`</code> function uses Bubble Sort, which is $O(N^2)$. While correct and acceptable for <code>`MAX_RECORDS = 100`</code> , it is not an optimized sorting algorithm for larger datasets. No other significant speed optimizations were implemented.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Basic new features such as <code>`display_account`</code> (search and display single account) and the <code>`get_positive_double`</code> helper function are well-implemented and enhance usability.
Requirement Translation	10	The student effectively translated requirements for new features (e.g., a dedicated debit process, sorted data export) into clear, functional code within the program's framework.
Added Functionality Simple	10	The <code>`debit_transaction`</code> function is a great example of simple, self-contained functionality, incorporating input validation and critical business logic (insufficient balance check).
Added Functionality Advanced	15	The <code>`write_sorted_text_file`</code> feature is the most advanced addition, involving reading multiple records, implementing a sorting algorithm (<code>`compare_names`</code> and Bubble Sort), and writing formatted output to a new file. This demonstrates a more complex understanding of data processing and file management.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UAM251

Repository: https://github.com/SARATHI-78/miniProjectSourceCode_251

Metric	Value
Commit Count	4
Public Repository	Yes
README Present	Yes

Final Score: 12.0 / 20

Overall Remarks: The student has made exceptional modifications to the banking system. They introduced a robust set of new features including a login system, comprehensive transaction logging, money transfer functionality with validation, interest application, and various ways to view account details and transaction history. The code demonstrates strong domain knowledge, excellent functional decomposition, and a clear understanding of C programming. The `logTransaction` utility is a particularly good addition, enhancing the system's accountability. While there isn't explicit optimization for memory or speed, the sheer volume and practical utility of the added functionalities make this an outstanding submission.

GENERAL

Criteria	Score	Remarks
Self Effort	5	The student has demonstrated significant effort by implementing a wide array of new functionalities and integrating them into the existing system. The volume and complexity of the additions are substantial.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Excellent understanding of basic banking operations, including money transfers, interest application, transaction logging, and statement generation. The implemented logic reflects core banking principles.
Added Functionality Ideas	5	The ideas for added functionalities are highly relevant and greatly enhance the utility and completeness of the banking system. Features like login, transaction history, and money transfer are crucial improvements.
Code Comprehension	5	The student's code is clear, well-structured, and easy to understand. New functions are logically defined and their purpose is evident, indicating strong comprehension of programming principles.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	The introduction of the <code>logTransaction</code> utility function, and its subsequent integration into various operations (update, create, delete, transfer, interest), significantly improves the system's auditability and accountability. This is a notable code improvement for the overall system.
Functional Decomposition	5	The new features are impeccably decomposed into distinct, well-defined functions (<code>transferMoney</code> , <code>logTransaction</code> , <code>transactionHistory</code> , <code>applyInterest</code> , <code>miniStatement</code> , <code>viewAccount</code>). This demonstrates excellent modular design.

Criteria	Score	Remarks
Memory Optimization	0	No explicit memory optimization techniques were observed in the added code. Standard file I/O and data structures are used.
Speed Optimization	0	No explicit speed optimization techniques were observed. Operations like <code>miniStatement`</code> using <code>strstr`</code> to search a file line-by-line could become slow with very large transaction history files, but this is typical for simple file-based operations.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Implemented essential new features such as a basic admin login system, a robust transaction logging mechanism with timestamps, and a function to view the complete transaction history. These are fundamental additions.
Requirement Translation	10	The student successfully translated complex banking requirements (like secure money transfers with validation, periodic interest application, and filtered mini-statements) into functional and reliable code.
Added Functionality Simple	10	A substantial amount of 'simple' but highly impactful functionality has been added, including money transfer with balance checks, automatic interest application, a detailed account viewing function, and a mini-statement feature. These significantly enhance the system's capabilities.
Added Functionality Advanced	5	While the breadth of features is impressive, the individual implementation of these features does not involve advanced algorithms, complex data structures beyond basic structs, or sophisticated error handling/security mechanisms typical of 'advanced' functionality in a broader computer science context. However, the comprehensive integration and interaction of these features (e.g., logging across multiple operations, filtering logs for mini-statements) demonstrate a good level of system-level thinking for a student project.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UCS149

Repository: <https://github.com/kavyaparthiban-ship-it/cminiproject>

Metric	Value
Commit Count	1
Public Repository	Yes
README Present	Yes

Final Score: 14.0 / 20

Overall Remarks: The student has done an exceptional job. They have not only added significant new features that greatly enhance the banking application but also implemented them with a high degree of correctness, robustness, and adherence to good coding practices. The code demonstrates a strong grasp of C programming, file I/O, domain knowledge, and problem-solving. The error handling and functional decomposition are particularly commendable.

GENERAL

Criteria	Score	Remarks
Self Effort	5	The student has demonstrated significant self-effort by adding three substantial new features, modifying the main program loop, updating the menu, and including a necessary header file. This goes beyond basic modifications.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Excellent understanding of banking domain knowledge. The 'transferFunds' function correctly handles source/destination accounts, checks for existence, validates sufficient funds, and updates both accounts. 'accountAudit' correctly calculates total deposits and identifies the richest client. 'searchByFullName' correctly uses both first and last names for searching. Error conditions are well-handled.
Added Functionality Ideas	5	The chosen functionalities (transfer funds, global account audit, search by name) are highly relevant and significantly enhance the banking system's utility. They reflect a good understanding of core requirements for such an application.
Code Comprehension	5	The added code is well-structured, readable, and uses clear variable names. The logic within each new function is easy to follow. Correct use of file I/O functions (fseek, fread, fwrite) and string comparison (strcmp) is evident.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	The student introduced significant improvements in robustness through input validation (e.g., checking for non-existent accounts, insufficient funds, positive transfer amounts) and better error messaging. The integration of new features is clean and does not introduce unnecessary complexity.

Criteria	Score	Remarks
Functional Decomposition	10	All new features are well-encapsulated in separate, dedicated functions ('transferFunds', 'accountAudit', 'searchByFullName'). This demonstrates excellent functional decomposition, keeping the 'main' function and 'switch' statement clean and organized.
Memory Optimization	5	The implementation correctly processes file records one by one, avoiding loading the entire file into memory. This is efficient for memory usage, especially for potentially large data files.
Speed Optimization	5	For operations like 'transferFunds', the code efficiently uses 'fseek' to directly access records. For 'accountAudit' and 'searchByFullName', linear iteration is necessary given the file structure, and it is implemented without unnecessary overhead.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Three distinct and fundamental banking features were added: fund transfers, account auditing, and searching by full name. These are practical and well-implemented.
Requirement Translation	10	The student effectively translated abstract requirements for banking functionalities into robust and concrete C code. This includes careful handling of file pointers, multi-record updates (for transfers), error checking, and user feedback.
Added Functionality Simple	5	The core concepts of the added functionalities are straightforward and well-executed, demonstrating competence in implementing essential system features.
Added Functionality Advanced	5	While the functionalities themselves are not 'advanced' algorithms, the robust implementation in C, particularly the 'transferFunds' function involving careful reading, modification, and writing back to two separate file records with comprehensive error handling, showcases an advanced understanding of file I/O and data consistency for a student project.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UCS247

Repository: https://github.com/Tarunika-Muruganathan/711725UCS247_CSE-B_C-Mini-project

Metric	Value
Commit Count	21
Public Repository	Yes
README Present	Yes

Final Score: 15.4 / 20

Overall Remarks: The student has significantly enhanced the base banking system by introducing robust features. Key additions include a multi-layered security system with admin login and PIN-based authentication for sensitive transactions, extended client data management, core banking operations (deposit, withdraw, transfer), and a transaction logging system for auditing. The functional decomposition is excellent, and the implementation demonstrates a strong understanding of banking domain requirements. While minor structural issues like unreachable code and missing braces for simple functions were observed, the overall contribution and logic are highly commendable, making the system much more functional and secure.

GENERAL

Criteria	Score	Remarks
Self Effort	5	Student added significant new features including admin login, detailed client fields, PIN authentication for sensitive operations, and core banking functionalities like deposit, withdraw, and transfer, along with transaction logging. This indicates substantial effort.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Demonstrates excellent understanding of banking system requirements including user authentication (admin and PIN), rich client data management, core transaction types (deposit, withdraw, transfer), and security considerations.
Added Functionality Ideas	5	All added functionalities are highly relevant and well-conceived for a banking system, addressing security, data richness, and core financial operations.
Code Comprehension	5	The student demonstrates strong comprehension of their added code, evident in the consistent application of PIN checks across multiple functions, proper integration of new functionalities, and clear error handling for new features.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	Significant code improvement through the addition of robust PIN authentication across multiple operations, admin login, and `isValidDOB` validation. The display logic is also enhanced with more client details. A minor issue with unreachable code in the `login()` function and some missing braces for simple function definitions was noted.

Criteria	Score	Remarks
Functional Decomposition	10	Excellent functional decomposition. Core banking operations (deposit, withdraw, transfer) are encapsulated in dedicated functions, along with helper utilities like `isValidDOB` and `clearBuffer`, and separate functions for authentication and transaction history.
Memory Optimization	0	No specific memory optimization techniques were identified in the provided modifications.
Speed Optimization	0	No specific speed optimization techniques were identified in the provided modifications.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Basic new features include extended client data fields (gender, DOB, nationality, account type) and a `isValidDOB` utility function for basic data validation.
Requirement Translation	10	Excellent translation of general banking requirements (e.g., security, transaction processing, data integrity, record keeping) into practical and functional code features.
Added Functionality Simple	9	Implemented crucial functionalities such as admin login and basic deposit/withdraw operations, which are fundamental for a banking system, complete with validation and transaction logging.
Added Functionality Advanced	18	Advanced functionalities include robust PIN-based authentication integrated across all sensitive operations (search, update, deposit, withdraw, transfer, delete), the complex money transfer logic between accounts (involving multiple reads/writes and checks), and the implementation of a transaction logging system for auditing purposes.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UEC163

Repository: <https://github.com/mutharasu-kannan/miniProjectSourceCode>

Metric	Value
Commit Count	7
Public Repository	Yes
README Present	Yes

Final Score: 17.6 / 20

Overall Remarks: The student has made extensive and highly valuable modifications to the base code. Key additions include a robust security system with password authentication and account locking, comprehensive transaction logging with timestamps, and core banking functionalities like fund transfers with daily limits and transaction history viewing. The code is well-structured, clean, and demonstrates an excellent understanding of both programming practices and domain-specific requirements. This is an outstanding submission, significantly enhancing the functionality and reliability of the banking application.

GENERAL

Criteria	Score	Remarks
Self Effort	5	Demonstrated exceptional self-effort by implementing a comprehensive suite of new features including robust security, transaction logging, and core banking functionalities.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Showed outstanding grasp of banking domain concepts, accurately translating real-world requirements for security, transaction management, and user alerts into the application design.
Added Functionality Ideas	5	Exhibited highly creative and practical ideas for new functionalities, transforming a basic account management system into a more secure and feature-rich banking application.
Code Comprehension	5	The code is exceptionally well-written, clear, and demonstrates a deep understanding of C programming concepts and best practices, including string handling and file I/O.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	Implemented significant code improvements by adding critical features like password protection, transaction logging, and balance alerts, elevating the system's overall quality and usefulness.
Functional Decomposition	10	Excellent functional decomposition, with new logic clearly separated into modular and reusable functions, improving code organization and maintainability.
Memory Optimization	4	Memory usage is efficient, employing standard C practices like <code>`strncpy`</code> to manage buffer sizes, indicating an awareness of good memory handling.

Criteria	Score	Remarks
Speed Optimization	4	Operations like file I/O are performed efficiently for a direct file access system. While a linear scan for transaction history is used, it's appropriate for the assumed scale of data.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Successfully introduced fundamental new features such as secure password handling, account locking for failed attempts, and balance alerts, which are crucial for a banking system.
Requirement Translation	10	Demonstrated exceptional ability to translate complex banking requirements, such as secure fund transfers with limits and detailed transaction logging, into well-implemented code.
Added Functionality Simple	10	Added essential functionality like fund transfers between accounts and comprehensive transaction history viewing, significantly expanding the system's capabilities.
Added Functionality Advanced	20	Introduced advanced functionalities including a robust security model with account lockout, detailed time-stamped transaction logging for auditing, and sophisticated fund transfer logic with built-in limits, showcasing a high level of innovation and system design.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UEC233

Repository: <https://github.com/sanjaykumars25ec-design/Sanjaysspointer>

Metric	Value
Commit Count	2
Public Repository	Yes
README Present	No

Final Score: 10.4 / 20

Overall Remarks: The student has demonstrated a strong capability in enhancing a banking application. The modifications show a clear understanding of domain knowledge, excellent translation of requirements into code, and a good effort in improving the user experience and robustness. Key contributions include the implementation of crucial business rules like minimum balance, comprehensive error handling for account management, and a practical feature to export data to a human-readable text file. The code is well-structured and follows good functional decomposition. While the features are not 'advanced' in a complex software sense, they are very well-executed and practical for the scope of the project.

GENERAL

Criteria	Score	Remarks
Self Effort	4	The student has put in good effort to implement several significant features and improvements beyond a basic template. The inclusion of business rules and reporting functionality demonstrates initiative.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Strong understanding of basic banking domain concepts, including account types, balances, and transaction rules. Successfully implemented a minimum balance requirement for both account creation and updates/withdrawals.
Added Functionality Ideas	5	Good ideas for added functionality, such as exporting data to a text file, implementing a minimum balance rule, and providing clear user feedback for account operations (found/not found, transaction successful).
Code Comprehension	5	The student demonstrates good comprehension of C programming constructs and file I/O for direct access binary files. The logic for each function is clear and appropriate for its intended task, indicating a solid understanding of how to manipulate data within the file structure.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	4	Significant improvements in user experience through well-formatted menus, clear account detail displays, and informative success/error messages. The addition of business rules (minimum balance) and account existence checks enhances the robustness of the application.

Criteria	Score	Remarks
Functional Decomposition	4	The code exhibits good functional decomposition. Operations like 'add account', 'update account', 'delete account', 'display account', 'create text file', and 'enter choice' are clearly separated into distinct, single-responsibility functions.
Memory Optimization	0	No explicit memory optimization techniques were identified in the provided snippets. The approach of using `struct clientData` for file I/O is standard and efficient for this context but doesn't show specific 'optimization' efforts.
Speed Optimization	0	No explicit speed optimization techniques were identified. Direct access file I/O (`fseek`) is inherently efficient for record-based operations, but this is a standard design choice rather than a student-added 'optimization'.

INNOVATION

Criteria	Score	Remarks
New Features Basic	4	Key basic new features include the implementation of a minimum balance rule for account creation and transactions, robust account existence checks for CRUD operations, and improved user interface elements like formatted output and detailed menus.
Requirement Translation	8	Excellent translation of implied banking requirements (e.g., 'accounts should have a minimum balance', 'users need a summary of their accounts') into concrete, functional code logic, such as the minimum balance checks and the 'Save to Text File' feature.
Added Functionality Simple	8	The 'Save all to Text File' functionality is a strong, simple addition, providing a useful reporting feature. The minimum balance checks across different operations (add, update) are also well-implemented simple functional enhancements.
Added Functionality Advanced	5	While the added functionalities are robust and significantly improve the application's utility and adherence to business rules, they do not venture into 'advanced' software engineering concepts such as complex algorithms, network communication, or multi-threading. They are solid enhancements to a foundational CRUD system.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UIT142

Repository: <https://github.com/kalaivani-venkatesan/Kalaivani-TPS>

Metric	Value
Commit Count	5
Public Repository	Yes
README Present	Yes

Final Score: 13.6 / 20

Overall Remarks: The student has demonstrated exceptional effort and skill in significantly enhancing the banking system. Key modifications include a robust security layer with PIN hashing and a 3-attempt login leading to account locking, implementation of daily withdrawal limits, a detailed transaction logging system with timestamps, and a well-structured administrative interface. The code shows good functional decomposition and an understanding of core banking requirements. The added functionalities greatly improve both the security and usability of the system.

GENERAL

Criteria	Score	Remarks
Self Effort	5	Demonstrated substantial effort by implementing a wide array of new features and security measures beyond a basic template.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Good understanding of banking domain concepts including account security (PIN hashing, login attempts, locking), transaction types, daily withdrawal limits, and administrative controls.
Added Functionality Ideas	5	Excellent range of ideas, including PIN hashing, a secure 3-attempt login with account locking, daily transaction limits, comprehensive transaction logging, receipt generation, and a structured admin panel with various options.
Code Comprehension	5	Solid understanding of C programming, including file I/O operations (especially using <code>fseek</code> and <code>fwrite</code> for updating binary records), and integrating various modules (account, transaction, admin, utils) effectively.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	4	Includes valuable code improvements such as input validation (<code>if (acc.balance < 0)</code> , <code>if (amt <= 0)</code>), enhanced security with PIN hashing and login attempt limits, and improved user experience with <code>clearScreen()</code> and <code>pauseScreen()</code> utilities.
Functional Decomposition	4	The code is well-decomposed into logical functions and implicitly into modules (account, transaction, admin, utils), showing good separation of concerns. Examples include <code>hashPin</code> , <code>logTransaction</code> , <code>generateReceipt</code> .

Criteria	Score	Remarks
Memory Optimization	2	No explicit memory optimization techniques are evident, but there are no obvious memory misuse issues. Standard data structures are used.
Speed Optimization	2	No specific speed optimization techniques are evident. File I/O operations are standard sequential access for binary files, which is typical for this kind of application.

INNOVATION

Criteria	Score	Remarks
New Features Basic	4	Basic but crucial new features like PIN hashing, an `isActive` flag for accounts, and initial input validations were successfully introduced.
Requirement Translation	8	Effectively translates common banking requirements (secure login, transaction tracking, daily limits, admin oversight) into functional code.
Added Functionality Simple	8	Implemented significant simple functionalities such as a daily withdrawal limit, comprehensive transaction logging with timestamps, and basic receipt generation. These add substantial value.
Added Functionality Advanced	16	Introduced advanced functionalities like a 3-attempt login system with automatic account locking, a structured admin panel with various management options (including 'Search by Name'), and implied complex features like 'Transfer' and 'Transaction History' display.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UIT219

Repository: <https://github.com/Ragavendhar25/miniProjectSourceCode>

Metric	Value
Commit Count	5
Public Repository	Yes
README Present	Yes

Final Score: 10.8 / 20

Overall Remarks: The student successfully implemented a new 'search record' feature by account number. This involved adding a dedicated function (`searchRecord`), integrating it into the main menu, and utilizing appropriate file I/O operations like `fseek` and `fread`. The implementation demonstrates a solid understanding of the existing codebase and file management principles. The new functionality is practical, well-structured, and enhances the overall utility of the application.

GENERAL

Criteria	Score	Remarks
Self Effort	4	The student made a clear effort to identify and implement a new, useful feature (account search) and integrate it into the existing system.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Demonstrated good understanding of file I/O operations (<code>fseek</code> , <code>fread</code>), fixed-size record structures, and how to identify empty records in the binary file.
Added Functionality Ideas	5	The idea of adding an account search functionality is practical and significantly enhances the usability of the record management system.
Code Comprehension	5	The student clearly understood the existing code's patterns for file operations, input handling, and record manipulation, successfully applying them to the new feature.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	2	While the new code for searching is well-structured and consistent with the existing style, there are no explicit improvements made to pre-existing code functionalities.
Functional Decomposition	8	The new search functionality is correctly encapsulated within its own function (<code>searchRecord</code>) and properly integrated into the main application's menu, demonstrating good functional decomposition.
Memory Optimization	0	No explicit memory optimizations were observed in the added code. Standard data structures and practices are used.
Speed Optimization	3	The search functionality efficiently uses <code>fseek</code> for direct access to records based on account number, providing $O(1)$ access time for this specific search type.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Adding a search function by account number is a fundamental and highly beneficial new feature for a record management application.
Requirement Translation	10	Successfully translated the implied requirement of being able to find specific account details into a robust and functional implementation, handling input, validation, and display.
Added Functionality Simple	7	The `searchRecord` function adds valuable functionality that is straightforward to implement given the fixed-size record structure and direct access method.
Added Functionality Advanced	0	The added search feature, while useful, does not incorporate advanced techniques or complex algorithms (e.g., searching by multiple criteria, dynamic string matching).

PLAGIARISM CHECK

Other Roll	Similarity
711725UIT222	0.80

Student Evaluation Report

Roll Number: 711725UIT222

Repository: <https://github.com/ravirajv25it/miniProjectSourceCode>

Metric	Value
Commit Count	6
Public Repository	Yes
README Present	Yes

Final Score: 9.8 / 20

Overall Remarks: The student demonstrated a strong understanding of C file I/O operations and efficiency. The implementation of 'clearInputBuffer' and its consistent use throughout the code significantly improves input robustness. The optimizations in file operations for updating, deleting, and creating records, particularly the intelligent use of 'fseek' with 'SEEK_CUR' and a global 'BLANK_CLIENT' constant, showcase a commendable effort in code improvement and efficiency. While no entirely new core features were implemented in the provided snippet (the proposed 'accounts.txt' feature was only mentioned in the menu string), the focus on making existing functionalities robust and performant is evident.

GENERAL

Criteria	Score	Remarks
Self Effort	4	The student demonstrated good effort in refining the base code by introducing a global blank client structure and a helper function for input buffer clearing, applying them consistently.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Strong understanding of C file I/O operations, including efficient use of fseek with SEEK_SET and SEEK_CUR, fread, and fwrite. Also shows knowledge of input buffer handling issues.
Added Functionality Ideas	2	The menu text explicitly mentions '2 - store a formatted text file of accounts called "accounts.txt" for printing,' which is an idea for added functionality, though no implementation is provided in the snippet.
Code Comprehension	5	The student's modifications, especially the optimized file operations and the clearInputBuffer function, are well-understood and logically implemented.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	Significant code improvements include the introduction of the BLANK_CLIENT global constant, the clearInputBuffer utility function, and particularly the optimized fseek/fwrite patterns in update, delete, and create functions which enhance efficiency and robustness.
Functional Decomposition	4	The addition of the 'clearInputBuffer' helper function is a good example of functional decomposition, encapsulating a common utility task.

Criteria	Score	Remarks
Memory Optimization	5	The use of 'static const struct clientData BLANK_CLIENT' as a global constant provides a minor but valid memory optimization by avoiding repeated stack allocation and initialization of a blank client structure within functions.
Speed Optimization	8	The 'OPTIMIZED' file operations demonstrate good speed optimization by intelligently using fseek with SEEK_CUR to avoid redundant seeks from the beginning of the file (SEEK_SET) when performing read-modify-write operations, thus reducing disk I/O overhead.

INNOVATION

Criteria	Score	Remarks
New Features Basic	0	No entirely new core features are implemented in the provided code snippet. The improvements focus on enhancing existing functionalities.
Requirement Translation	7	The student effectively translated implicit requirements for robust input handling (clearInputBuffer) and efficient file I/O (optimized fseek/fwrite operations) into well-crafted code.
Added Functionality Simple	4	The 'clearInputBuffer' function is a simple, yet crucial, piece of added utility functionality that improves the overall usability and robustness of the application's input processing.
Added Functionality Advanced	0	No advanced functionality or complex feature additions are present in the provided code snippet.

PLAGIARISM CHECK

Other Roll	Similarity
711725UIT219	0.80